二、1、直线扫描算法

DDA算法（Digital Differential Analyzer）

- 当直线斜率小于1时，从x的左端点开始，向x右端点步进，步长为1，以此计算出y的坐标，并取（x，int(y+0.5)）为当前点的坐标。当直线斜率绝对值大于1，以y轴作为步进方向

中点画线法

假定直线斜率 k 在 0~1 之间，当前像素点为（x_p，y_p），则下一个像素点为 p_1（$x_p + 1$，y_p）或 p_2（$x_p + 1$，$y_p + 1$）。设 p_1和 p_2的中点为 M（$x_p + 1$，$y_p + 0.5$），Q 为理想直线和 $x = x_p + 1$ 的垂线的交点。当 M 在 Q 的上面时，取 p_1为下一像素点；反之，p_2为下一像素点。

Bresenham算法

- 过各行、各列像素中心构造一组虚拟网格线，按直线从起点到终点的顺序计算直线与各垂直网格线的交点，然后确定该列像素中与此交点最近的像素。

Bresenham算法程序实现：

```
void Bresenhamline(int x0, int y0, int x1, int y1, int color) {
    int x, y, dx, dy, p;
    dx = x1 - x0; dy = y1 - y0;
    x = x0; y = y0;
    if (dy < 0) { dy = -dy; dx = -dx; }
    int i = 2 * dy - dx; p = 2 * dy;
    for (; x <= x1; x++) {
        drawpixel(x, y, color);
        if (i < 0) { i += 2 * dy; } else {
            if (i > 0) { y++; }
            i += 2 * (dy - dx);
        }
    }
}
```

二、2、中点画线法转换直线段

给定点A(1, 0)和点B(4, 7)，使用中点画线法转换直线段，并给出每一步的判别式。

- $$初始点：(x_0 = 1, y_0 = 0)$$
- $$终点：(x_1 = 4, y_1 = 7)$$
- $$斜率：(k = \frac{y_1 - y_0}{x_1 - x_0} = \frac{7}{3})$$
-

判别式计算

- $$\begin{aligned} a &= x_0 - x_1 = -3 \\ b &= y_1 - y_0 = 7 \\ c &= x_1 \cdot y_0 - x_0 \cdot y_1 = -7 \end{aligned}$$

$$
\begin{aligned}
&d = F = F(y_p + 1, x_p + 0.5) = a(y_p + 1) + b(x_p + 0.5) + c \\
&2d = 2ay_p + 2bx_p + 2a + b + 2c
\end{aligned}
$$

计算过程：

1.

i	y_i	x_i	2d
1	0	1	2a + 3b + 2c = -6 + 21 - 14 = 1
2	1	1	4a + 3b + 2c = -12 + 21 - 14 = -5（d 小于 0，所以取 p_2）
3	2	2	6a + 5b + 2c = -18 + 35 - 14 = 3
4	3	2	8a + 5b + 2c = -24 + 35 - 14 = -3（）
5	4	3	10a + 7b + 2c = -30 + 49 - 14 = 5
6	5	3	12a + 7b + 2c = -36 + 49 - 14 = -1（）
7	6	4	14a + 9b + 2c = -42 + 63 - 14 = 7
8	7	4	16a + 9b + 2c = -48 + 63 - 14 = 1

二、4、字符串精度

字符串精度可以分为以下三个精度：

1. **串精度**：将包围字串的外接矩形对窗口作裁剪，当字符串方框整个在窗口内时予以显示，否则不显示。
2. **字符精度**：将包围字的外接矩形对窗口作裁剪，当字符方框整个在窗口内时予以显示，否则不显示。
3. **笔画/象素精度**：将矢量字符的笔划分解成直线段对窗口作裁剪；将点阵字符像素相对窗口边界作取舍，当字符方框整个在窗口内时予以显示，否则不显示。

二、5、字符类型及其特点

字符分为两种类型：

1. **点阵字符**：存储量大，易于表示。
2. **矢量字符**：存储量小，美观，变换方便；但需要光栅化（将几何数据经过一系列变换最终转换为像素）后才能显示。

二、8、走样和反走样

走样：由于显示器的空间分辨率有限，因像素逼近误差，使所画图形产生畸变（台阶、锯齿）的现象（用离散量表示连续量引起的失真）。

反走样：用于减少或消除这种效果的技术。

反走样的主要方法：

1. **提高分辨率**：显示器分辨率提高，直线经过的像素加倍，锯齿也加倍，同时每个阶梯的宽度减少，显示出来的直线就更加平滑。只能减轻而不能消除锯齿现象。

2. **区域采样**：假定每个像素是具有一定面积的小区域，将直线段看做具有一定宽度的狭长矩形，当直线段和像素有交时，求出两者的相交区域面积，然后根据相交区域面积的大小确定该像素的亮度值。
3. **加权区域采样**：在区域采样的基础上，使相交区域对像素亮度的贡献依赖于该区域与像素中心的距离。

三、3、Bezier曲线合并条件

Error: Missing close brace

设一条二次Bezier曲线的控制顶点为(P_0, P_1, P_2)，另一条Bezier曲线的顶点是(Q_0, Q_1, Q_2)。$(P_2 = Q_0)$。两条曲线可以精确合并（表示）为一条二次Bezier曲线的条件：

1. $$(P_1, P_2(Q_0), Q_1)\text{三点共线。}$$
2. 根据抛物线三切线定理：

$$\frac{Q_2 - Q_1}{Q_1 - Q_0} = \frac{Q_1 - Q_0}{Q_0 - P_1} = \frac{P_2 - P_1}{P_1 - P_0}$$

$Q_1 - \frac{Q_2 - Q_1}{Q_1 - Q_0}(Q_0 - P_1) = P_1 - \frac{P_1 - P_0}{P_2 - P_1}(Q_1 - Q_0)$

三、4、Bezier曲线控制顶点反求

已知Bezier曲线上的4个点分别为$(Q_0(50,0), Q_1(100,0), Q_2(0,50), Q_3(0,100))$，它们对应的参数分别为0，1/3，2/3，1，反求Bezier曲线的控制顶点。给出4次Bezier曲线的控制顶点。给出4次Bezier曲线退化为3次Bezier，控制顶点(P_0, P_1, P_2, P_3)应该满足的条件。

反求控制顶点：

$$(P_0 = Q_0 = (50,0))$$
$$(P_3 = Q_3 = (0,100))$$

根据Bezier曲线的公式：

$$Q_1 : \frac{8}{27}P_0 + \frac{4}{9}P_1 + \frac{2}{9}P_2 + \frac{1}{27}P_3 = (100,0)$$
$$Q_2 : \frac{1}{27}P_0 + \frac{2}{9}P_1 + \frac{4}{9}P_2 + \frac{8}{27}P_3 = (0,50)$$

求解得：

$$P_1 = \left(\frac{775}{3}, -\frac{125}{3}\right), P_2 = \left(-\frac{400}{3}, \frac{200}{3}\right)$$

退化条件：

$$\Delta_3 P_0 = \sum_{i=0}^{3}(-1)^i C_3^i P_{3-i} = 0$$

三、5、Bezier曲线形状调整方法

设一条三次Bezier曲线的控制顶点为

$$(P_0, P_1, P_2, P_3)$$

。对曲线上一点(P(0.5))，及一个给定的目标点(T)，给出一种调整Bezier曲线形状的方法，使得(P(0.5))精确通过点T*T*。

假设我们改变其中的一个控制顶点，比如把P_1改成$(P_1 + \lambda)$，使得$P(0.5)$精确通过点T，将修改后的曲线记为$(\widehat{P}(t))$，则：

$[\widehat{P}(t)| * \frac{1}{2} = \sum *t = 0^3 P_i B_{i,3}(t)| * \frac{1}{2} + \lambda B * 1, 3(\frac{1}{2})]$

即$(T = P(\frac{1}{2}) + \lambda B_{1,3}(\frac{1}{2}))$，

求得$(\lambda = (T - P(\frac{1}{2})/B_{1,3}(\frac{1}{2})))$，

所以调整方法为：把(P_1)调整到$(P_1 + (T - P(\frac{1}{2}))/B_{1,3}(\frac{1}{2}))$。

三、6、4次Bezier曲线计算与de Casteljau三角形

计算以(30,0),(60,10),(80,30),(90,60),(90,90)(30,0),(60,10),(80,30),(90,60),(90,90)为控制顶点的4次Bezier曲线在t=1/2*t*=1/2处的值，并画出de Casteljau三角形。

de Casteljau三角形：

P_0(30,0) ↘️
P_1(60,10) ↘️ ➡️ P_0^1(45,5) ↘️
P_2(80,30) ↘️ ➡️ P_1^1(70,20) ↘️ ➡️ P_0^2(57.5,12.5) ↘️
P_3(90,60) ↘️ ➡️ P_2^1(85,45) ↘️ ➡️ P_1^2(77.5,32.5) ↘️ ➡️ P_0^3(67.5,22.5) ↘️
P_4(90,90) ➡️ P_2^1(70,75) ➡️ P_2^2(87.5,60) ➡️ P_1^3(82.5,46.25) ➡️ P_0^4(75,34.375)

三、7、Bezier曲线升阶公式

推导Bezier曲线的升阶公式；给定三次Bezier曲线的控制顶点(0,0),(0,100),(100,0),(100,100)(0,0),(0,100),(100,0),(100,100)，计算升阶一次后的控制顶点。

设给定原始控制顶点$P_0, P_1, \ldots, P_n$，定义了一条 n 次 Bezier 曲线：

$$P(t) = \sum_{i=0}^{n} P_i B_{i,n}(t) \qquad t \in [0,1]$$

增加一个顶点，仍定义同一条曲线的新控制顶点为$P_0^*, P_1^*, \ldots, P_{n+1}^*$，则有：

$$\sum_{i=0}^{n} P_i C_n^i t^i (1-t)^{n-i} = \sum_{i=0}^{n+1} P_i^* C_{n+1}^i t^i (1-t)^{n+1-i}$$

对上式左边乘以（t + （1 - t）），得到：

$$\sum_{i=0}^{n} P_i C_n^i (t^{i+1}(1-t)^{n-i} + t^i (1-t)^{n+1-i}) = \sum_{i=0}^{n+1} P_i^* C_{n+1}^i t^i (1-t)^{n+1-i}$$

对比左右两式$t^i(1-t)^{n+1-i}$的系数，得：

$$P_i^* C_{n+1}^i = P_i C_n^i + P_{i-1} C_n^{i-1}$$

化简得：

$$P_i^* = (1 - \frac{i}{n+1}) P_i + \frac{i}{n+1} P_{i-1}$$

- **计算升阶一次后的控制顶点（本题n = 3）**：

原始	升阶
P_0(0,0)	P_0*(0,0)
P_1(0,100)	P_1*(0,75)
P_2(100,0)	P_2*(50,50)
P_3(100,100)	P_3*(100,25)
P_4*(100,100)	

三、9、Bezier曲线退化条件

n 次 Bezier 曲线：

$$P(t) = \sum_{i=0}^{n} P_i B_{i,n}(t) = \sum_{i=0}^{n} P_i C_n^i t^i (1-t)^{n-i}$$

可知t^n系数为：

$$\sum_{i=0}^{n} (-1)^i C_n^{n-i} P_{n-i} = \sum_{i=0}^{n} (-1)^i C_n^i P_{n-i}$$

根据公式：

$$\Delta^k P_i = \Delta(\Delta^{k-1} P_i) = \Delta^{k-1} P_{i+1} - \Delta^{k-1} P_i = \sum_{i=0}^{k} (-1)^i C_k^i P_{k-i}$$

所以可得：

$$\Delta^n P_0 = \sum_{i=0}^{n} (-1)^i C_n^i P_{n-i}$$

n 次 Bezier 曲线退化为 n - 1 次 Bezier 曲线条件是 t 的次数最高项t^n系数等于 0，所以原式$\Delta^n P_0 = 0$得证。

三、14、形体表示方法

常见的形体表示方法有：

1. **线框模型**：只能表示一些简单模型。
2. **实体模型**：完整的、无歧义地表示三维形状，不适合物体表面的运算：关照计算、阴影计算和光线求交计算。
3. **表面模型**：基于连续参数曲面的表示方法（如样条等）和离散表面表示方法（如三角网格）。容易表达工业中较光滑的曲线曲面，难以表达真实世界的物体。